

UGV Navigation with Dynamic and Unknown Obstacles

1. Problem Statement

In the previous problem, we assumed that **all obstacles were static and known beforehand**. This assumption is not realistic in a real-world environment.

In a real battlefield or outdoor environment:

- Some obstacles may **move**.
- New obstacles may suddenly appear.
- Some obstacles may not be known before the UGV starts moving.
- Previously free paths may become blocked.
- The UGV must continuously observe its surroundings and change its path when necessary.

Therefore, the UGV needs a **dynamic path-planning approach** that can detect obstacles and **re-plan its route while moving**.

A suitable approach is to use **A*** together with **real-time obstacle detection and repeated re-planning**. For larger or frequently changing environments, algorithms such as **D* Lite** or **Lifelong Planning A*** are also commonly used.

2. Aim

To design a navigation algorithm that enables a UGV to reach a user-defined goal while **detecting unknown dynamic obstacles, avoiding collisions, and continuously re-planning an optimal or shortest currently-known path**.

3. Difference from the Previous Problem

Previous problem

```
Known map
↓
Known obstacles
↓
Run A*
↓
Find path
↓
Follow path
```

↓
Reach goal

The environment does not change.

Dynamic-obstacle problem

Partial/unknown map

↓
Detect surroundings

↓
Update map

↓
Plan path

↓
Move UGV

↓
Detect new obstacle

↓
Update map again

↓
Re-plan

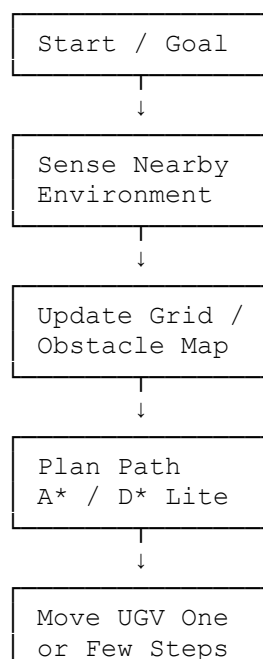
↓
Continue

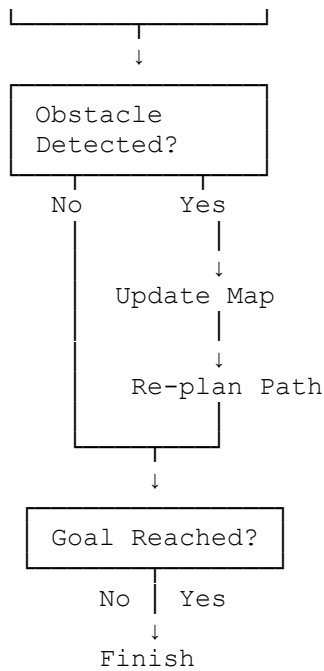
↓
Reach goal

The important change is that **path planning is no longer performed only once.**

4. Proposed Approach

The UGV can use a **Sense → Update → Plan → Move → Repeat** strategy.





5. Sensing the Environment

Since the obstacles are **not known a priori**, the UGV needs sensors to discover them.

Examples include:

- LiDAR
- Cameras
- Radar
- Ultrasonic sensors
- Depth cameras
- GPS/IMU for localization

For this assignment, we can simplify the sensor by assuming that the UGV can detect obstacles within a certain radius.

For example:

```

? ? ? ? ?
? . . . ?
? . U . ?
? . . . ?
? ? ? ? ?
  
```

Here:

- U = UGV
- . = currently known free space
- ? = unknown area

When the UGV moves, it gradually discovers the unknown area.

6. Dynamic Grid Representation

Instead of using only:

```
0 = free
1 = obstacle
```

we can use three states:

```
0 = Free
1 = Obstacle
2 = Unknown
```

For example:

```
? ? ? ? ? ? ?
? 0 0 0 1 ? ?
? 0 U 0 0 ? ?
? 0 0 0 0 ? ?
? ? ? ? ? ? ?
```

Initially, the UGV does not know the complete environment.

As it senses the surroundings:

```
Unknown → Free
```

or

```
Unknown → Obstacle
```

7. Dynamic Obstacles

There are two important types of obstacles.

Type 1: Previously Unknown Obstacle

The UGV is moving toward the goal:

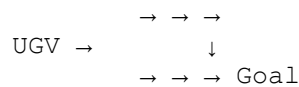
```
UGV → → → → Goal
```

It detects an obstacle:

```
UGV → → ■ → Goal
```

The original path is no longer valid.

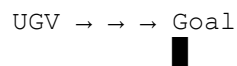
The UGV must calculate another route:



Type 2: Moving Obstacle

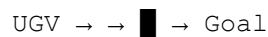
An obstacle can change its position:

Time 1:



Later:

Time 2:



The UGV needs to react to this change.

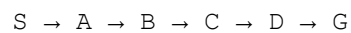
8. Re-planning

This is the most important part of the dynamic environment.

The UGV does **not blindly follow its original path**.

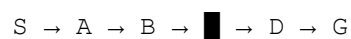
Instead, it repeatedly checks whether the current path is still safe.

Suppose the original path is:



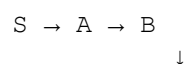
The UGV reaches B.

A new obstacle appears at C:



The UGV detects the obstacle and performs path planning again.

The new path might be:



$$E \rightarrow F \rightarrow G$$

Thus, the UGV can adapt to changes in the environment.

9. Using A* for Re-planning

A simple solution for the assignment is:

Run A* again whenever a new obstacle is detected on or near the current path.

The procedure becomes:

Step 1

Initialize the UGV position and goal.

Step 2

Sense the surrounding environment.

Step 3

Update the obstacle map.

Step 4

Run A* using the currently known map.

Step 5

Get the shortest currently-known path.

Step 6

Move the UGV one or several cells along that path.

Step 7

Sense the environment again.

Step 8

If a new obstacle is detected:

STOP

↓

UPDATE MAP
↓
REPLAN

Step 9

Continue until the goal is reached.

10. Why Not Just Run A* Once?

Because the original path may become invalid.

For example:

Original:

S → → → → G

A new obstacle appears:

S → → ■ → → G

If the UGV continues following the original path, it will collide with the obstacle.

Therefore:

Dynamic Environment ⇒ Continuous Re-planning
--

11. Better Algorithm: D* Lite

For a more realistic dynamic environment, **D* Lite** is a strong choice.

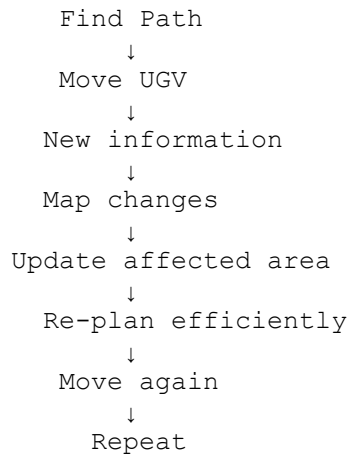
D* Lite is designed for situations where:

- The environment is initially unknown or only partially known.
- New information is discovered during navigation.
- Obstacles can change.
- The path needs to be updated efficiently.

Instead of calculating everything from scratch after every change, D* Lite can **reuse information from previous searches** and update the affected parts of the path.

Conceptually:

Initial Map
↓



12. A* vs D* Lite

Feature	A* Re-planning	D* Lite
Static obstacles	Yes	Yes
Unknown obstacles	Yes, with updates	Yes
Dynamic changes	Yes, by re-planning	Yes
Simple to implement	Easier	More complex
Reuses previous search information	Limited	Yes
Suitable for assignment demonstration	Very suitable	Suitable for advanced implementation
Suitable for continuously changing maps	Can become expensive	Designed for efficient updates

For a **simple academic implementation**, repeated A* is easier to understand and demonstrate. For a more advanced dynamic-navigation system, **D* Lite** is a natural extension.

13. Handling Moving Obstacles

For moving obstacles, merely knowing their current position may not be enough.

The UGV can maintain information such as:

```

Obstacle position
Obstacle velocity
Obstacle direction
  
```

For example:

```

Obstacle:
Current position = (20,30)
  
```

Velocity = (1,0)

This means the obstacle is moving approximately one cell horizontally per time step.

The UGV can estimate:

$$P_{future} = P_{current} + V\Delta t$$

and avoid cells where a collision is predicted.

14. Collision Checking

Before moving to the next cell, the UGV checks:

Is the next cell safe?

If:

Safe → Move

If:

Obstacle detected → Stop and Re-plan

Therefore:

Move only after checking the next location

This makes the navigation more robust.

15. Complete Algorithm

Dynamic UGV Navigation Algorithm

START

Set UGV position

Set GOAL position

Create partially known map

WHILE UGV has not reached GOAL:

 Sense the environment

```

Detect new obstacles

Detect changes in existing obstacles

Update obstacle map

Check whether current path is still safe

IF current path is blocked:

    Re-plan path using A*
    OR update path using D* Lite

IF path is available:

    Move UGV to next safe cell

ELSE:

    Stop

    Search for another feasible route

    IF no route exists:
        Report failure

END WHILE

Report successful navigation

END

```

16. Measures of Effectiveness

The same MOEs from the previous experiment can be used, but additional measures become important for a dynamic environment.

1. Path Length

Total distance travelled by the UGV.

$$PL = \sum_{i=1}^n d_i$$

2. Execution Time

Time required by the path-planning algorithm.

3. Nodes Expanded

Number of grid cells examined during planning.

4. Number of Re-plans

This is especially important for dynamic environments.

For example:

Initial planning → 1
Obstacle detected → 2
Another obstacle → 3
Another change → 4

So:

$$\text{Re-plans} = 4$$

A large number of re-plans indicates that the environment is changing frequently.

5. Collision Rate

The UGV should avoid collisions.

$$\text{Collision Rate} = \frac{\text{Number of collisions}}{\text{Total trials}}$$

Ideally:

$$\text{Collision Rate} = 0$$

6. Success Rate

$$\text{Success Rate} = \frac{\text{Successful missions}}{\text{Total missions}} \times 100$$

7. Path Efficiency

$$\text{Path Efficiency} = \frac{\text{Shortest possible distance}}{\text{Actual distance travelled}} \times 100$$

8. Re-planning Time

Measure how long the UGV takes to generate a new path after discovering an obstacle.

This is important because the UGV should react quickly to environmental changes.

17. Example Scenario

Consider a small grid:

```
S . . . . .
. . . . X . .
. . . . . . .
. . . X . . .
. . . . . . G
```

Initially, A* finds:

```
S → → → → → ↓
                ↓
                → → G
```

The UGV starts moving.

Suddenly, a new obstacle appears:

```
S → → → X → ↓
                ↓
                → → G
```

The UGV detects the obstacle.

It does not continue forward.

Instead:

```
Detect obstacle
      ↓
Stop
      ↓
Update map
      ↓
Run path planner
      ↓
Find alternative path
      ↓
Continue movement
```

The new path may go around the obstacle:

```
S → → ↓
      ↓
      ↓
      ↓ → → → G
```

This process continues until the UGV reaches the goal.

18. Improved Architecture

The complete system can be divided into five modules:

Module 1 – Environment Module

Creates and maintains the grid.

Module 2 – Sensing Module

Detects obstacles around the UGV.

Module 3 – Mapping Module

Updates the known map based on sensor information.

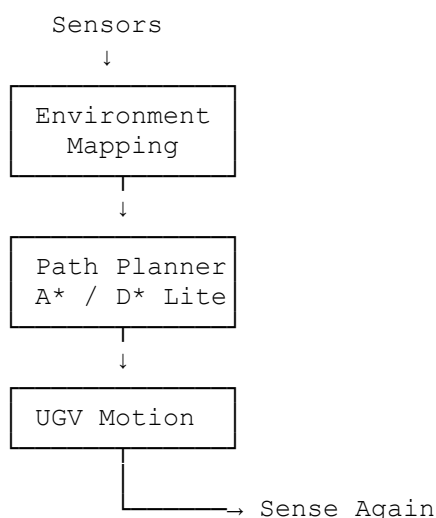
Module 4 – Path Planning Module

Uses:

- A* for simple re-planning, or
- D* Lite for more efficient dynamic re-planning.

Module 5 – Navigation Module

Moves the UGV along the calculated path and continuously checks for new obstacles.



19. Advantages of the Dynamic Approach

1. **Handles unknown obstacles.**
 2. **Adapts to changing environments.**
 3. **Reduces the chance of collision.**
 4. **Can find alternative routes.**
 5. **Does not require complete prior knowledge of the environment.**
 6. **Can continuously update the navigation plan.**
-

20. Limitations

1. Sensor information may be incomplete.
 2. Sensors may contain noise or errors.
 3. Very fast-moving obstacles are difficult to predict.
 4. Frequent re-planning can increase computation time.
 5. The shortest path at one moment may not remain the shortest later.
 6. Real UGV movement has physical constraints that a simple grid model does not represent.
-

21. Final Observation

In a dynamic environment, the UGV cannot depend on a single path calculated at the beginning. It must continuously **sense the environment, update its map, check the safety of its current path, and re-plan whenever necessary.**

A simple implementation can use **A* with repeated re-planning**. For a more advanced implementation where the environment changes frequently, **D* Lite** can efficiently update the path using information from previous searches.

Thus, the basic principle changes from:

"Plan once and move."

to:

"Sense → Plan → Move → Detect → Re-plan → Move → Repeat."

This makes the navigation system much more suitable for a **real-world UGV operating in an unknown and dynamic environment.**